

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Khoa Công nghệ Thông tin

BÁO CÁO BÀI TẬP

Môn: Nhập môn Công nghệ số và Trí tuệ nhân tạo

**KỸ NĂNG VIẾT PROMPT HIỆU QUẢ
CHO CÁC MÔ HÌNH NGÔN NGỮ LỚN**

(Thử nghiệm và phân tích 3 tác vụ học tập với ChatGPT)

Sinh viên thực hiện: Trần Anh Đức

Mã số sinh viên: 25021739

Ngày nộp: 30/03/2026

Hà Nội, 2026

1. Giới thiệu

Prompt engineering – kỹ năng viết câu lệnh hiệu quả cho các mô hình ngôn ngữ lớn (LLM) – đang trở thành một kỹ năng thiết yếu trong học tập và làm việc. Cùng một công cụ AI, cách đặt câu hỏi khác nhau có thể dẫn đến kết quả chất lượng rất khác nhau. Báo cáo này thử nghiệm và phân tích hiệu quả của 3 cấp độ prompt (cơ bản, cải tiến, nâng cao) trên 3 tác vụ học tập phổ biến, sử dụng ChatGPT (GPT-4o) làm công cụ thử nghiệm.

1.1. Ba tác vụ học tập được chọn

Tác vụ 1 – Tóm tắt tài liệu học thuật: Tóm tắt một bài báo khoa học về ứng dụng machine learning trong y tế. Thách thức: AI cần phân biệt ý chính và ý phụ, giữ đúng thuật ngữ chuyên ngành, không bóp méo kết luận của tác giả.

Tác vụ 2 – Giải thích khái niệm phức tạp: Giải thích khái niệm "Gradient Descent" trong machine learning. Thách thức: AI cần điều chỉnh mức độ chuyên sâu phù hợp với người đọc, sử dụng ví dụ trực quan mà không làm mất tính chính xác kỹ thuật.

Tác vụ 3 – Tạo bộ câu hỏi ôn tập: Tạo bộ câu hỏi ôn tập cho chương "Cấu trúc dữ liệu và giải thuật" (stack, queue, linked list). Thách thức: AI cần tạo câu hỏi đa dạng về mức độ (nhớ, hiểu, vận dụng), đúng nội dung và có đáp án chính xác.

2. Xây dựng các phiên bản prompt

2.1. Tác vụ 1 – Tóm tắt tài liệu học thuật

Cấp độ	Prompt
Cơ bản	"Tóm tắt bài báo này cho tôi."
Cải tiến	"Tóm tắt bài báo khoa học sau đây trong khoảng 200 từ. Nêu rõ: (1) mục tiêu nghiên cứu, (2) phương pháp sử dụng, (3) kết quả chính, (4) hạn chế. Sử dụng tiếng Việt, giữ nguyên các thuật ngữ chuyên ngành bằng tiếng Anh. [Nội dung bài báo]"
Nâng cao	"Bạn là một nghiên cứu sinh năm 3 ngành Khoa học máy tính, đang viết literature review. Hãy tóm tắt bài báo sau theo format: [Mục tiêu] → [Phương pháp] → [Kết quả] → [Hạn chế] → [Ý nghĩa cho nghiên cứu tương lai]. Độ dài: 200–250 từ. Tiếng Việt, giữ thuật ngữ tiếng Anh. Ví dụ output mong muốn: 'Nghiên cứu đề xuất mô hình XYZ để giải quyết bài toán ABC, sử dụng phương pháp...' [Nội dung bài báo]"

Kỹ thuật áp dụng ở prompt nâng cao: Role prompting (gán vai nghiên cứu sinh), structured output (format rõ ràng), few-shot example (ví dụ output mong muốn).

ChatGPT

Get Plus

Share

2603.26632v1.pdf

Tóm tắt bài báo này cho tôi.

Dưới đây là bản tóm tắt ngắn gọn, đúng trọng tâm của bài báo:

Mục tiêu bài báo

Nghiên cứu khả năng **tổng quát hóa (transferability)** của các mô hình Machine Learning trong phát hiện **malware**, đặc biệt khi:

- Dữ liệu đến từ nhiều nguồn khác nhau
- Có **obfuscation** (làm rối mã) và **concept drift** (thay đổi theo thời gian)

Vấn đề chính

- Các dataset malware hiện nay **không tương thích đặc trưng (features)** với nhau
- Mô hình học từ dataset này **khó áp dụng sang** $\downarrow$ **set khác**
- Malware ngày càng **biến đổi** và **che giấu** tốt hơn, làm giảm hiệu quả phát hiện

Kết quả chính

Hiệu quả tốt nhất

- Boosting models (đặc biệt **LightGBM**) hoạt động tốt nhất
- XGBFS > PCA trong giảm chiều
- Kích thước đặc trưng **384 chiều** cho kết quả tốt nhất

Đạt:

- AUC $\approx$ 99.8%
- F1 $\approx$ 98%

Vấn đề phát hiện

- Mô hình hoạt động tốt trên dữ liệu tương tự (TRITIUM, INFERNO)
- Nhưng **giảm mạnh hiệu suất** trên:
 - SOREL-20M (dữ liệu lớn, khác phân phối)
 - ERMDS (nhiều obfuscation)

Nguyên nhân:

- Distribution shift** (khác phân phối dữ liệu)
- Obfuscation** làm thay đổi đặc trưng
- Concept drift** theo thời gian

Phương pháp

- Dữ liệu**
Sử dụng nhiều dataset lớn (EMBER, BODMAS, SOREL-20M, ERMDS, TRITIUM, INFERNO)
- Pipeline xử lý**
 - Chuẩn hóa đặc trưng theo EMBER-v2
 - Scaling dữ liệu (Robust + MinMax)
 - Giảm chiều:
 - PCA
 - XGBoost Feature Selection (XGBFS)
- Mô hình**
 - LightGBM, XGBoost, Random Forest, Extra Trees
 - Dùng AutoML (FLAML) để tối ưu
 - Kết hợp 2 mô hình bằng **soft voting**
- Đánh giá**
 - F1-score
 - AUC
 - TPR tại mức lỗi thấp (0.1% và 1% FPR)

The image shows a screenshot of the ChatGPT web interface. At the top, the header says "ChatGPT" with a dropdown arrow. Below the header, there's a search bar with a magnifying glass icon. The main content area shows a conversation. The user's message is: "Tóm tắt nội dung trên thành 1 đoạn ngắn gọn, súc tích, dễ hiểu." The assistant's response is:

Kết luận

- ML có thể phát hiện malware hiệu quả, nhưng:
 - Không dễ tổng quát hóa giữa các dataset
- Hiệu quả phụ thuộc mạnh vào:
 - Dữ liệu huấn luyện
 - Cách xử lý đặc trưng
- Obfuscation là thách thức lớn

Hướng phát triển

- Nghiên cứu mô hình mạnh hơn (deep learning)
- Xử lý tốt hơn:
 - Concept drift
 - Obfuscation
- Cải thiện khả năng **generalization**

Tóm lại (1 câu)

Mô hình ML phát hiện malware rất mạnh trên dữ liệu quen thuộc, nhưng vẫn yếu khi gặp dữ liệu mới hoặc bị làm rối, nên vấn đề "tổng quát hóa" vẫn chưa có giải pháp quyết tốt.

Below the response, there's a search bar with the text "Ask anything" and a microphone icon. At the bottom, there's a footer that says "ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#)."

The screenshot shows a ChatGPT chat window. At the top, the header says "ChatGPT" with a dropdown arrow. On the right, there are icons for "Share" and a menu. The chat history on the left includes icons for a new chat, a search icon, and a list of previous chats. The main chat area contains a user message (in a light blue bubble) asking for a summary of a paper on malware detection and model performance, and a corresponding assistant response (in a white bubble). The response discusses the paper's focus on transferability, the datasets used (EMBER-v2, BODMAS, ERMDs, SOREL-20M, TRITIUM, INFERNO), the models tested (Robust Scaling, MinMax Scaling, PCA, XGBoost, Feature Selection, FLAML), and the results (AUC ≈ 99.8%, F1 ≈ 98%). It also mentions the paper's limitations regarding distribution shift and obfuscation. At the bottom, there is a text input field with the placeholder "Ask anything" and a microphone icon.

Bạn là một nghiên cứu sinh năm 3 ngành Khoa học máy tính, đang viết literature review. Hãy tóm tắt bài báo sau theo format: [Mục tiêu] → [Phương pháp] → [Kết quả] → [Hạn chế] → [Ý nghĩa cho nghiên cứu tương lai]. Độ dài: 200-250 từ. Tiếng Việt, giữ thuật ngữ tiếng Anh. Ví dụ output mong muốn: "Nghiên cứu đề xuất mô hình XYZ để giải quyết bài toán ABC, sử dụng phương pháp..." [Nội dung bài báo]

Nghiên cứu này nhằm đánh giá khả năng **transferability** của các mô hình Machine Learning trong bài toán **malware detection**, đặc biệt khi đối mặt với **distribution shift**, **obfuscation** và **concept drift** giữa các dataset khác nhau. Tác giả tập trung vào việc kiểm tra liệu các mô hình huấn luyện trên một tập dữ liệu có thể tổng quát hóa tốt sang các tập dữ liệu khác hay không.

Về phương pháp, nghiên cứu xây dựng một pipeline dựa trên đặc trưng chuẩn **EMBER-v2**, kết hợp nhiều dataset (EMBER, BODMAS, ERMDs, SOREL-20M, TRITIUM, INFERNO). Dữ liệu được tiền xử lý bằng **Robust Scaling** và **MinMax Scaling**, sau đó giảm chiều bằng **PCA** hoặc **XGBoost Feature Selection (XGBFS)**. Các mô hình như **LightGBM**, **XGBoost**, **Random Forest** và **Extra Trees** được huấn luyện với tối ưu siêu tham số bằng **FLAML**, và kết hợp thông qua **soft voting**.

Kết quả cho thấy các mô hình boosting (đặc biệt LightGBM) kết hợp với XGBFS ở 384 chiều đạt hiệu năng rất cao (AUC = 99.8%, F1 = 98%) trên dữ liệu cùng phân phối. Tuy nhiên, hiệu năng suy giảm rõ rệt khi đánh giá trên các dataset khác, nhất là với dữ liệu có obfuscation hoặc khác biệt thời gian như SOREL-20M và ERMDs.

Hạn chế chính là khả năng tổng quát hóa còn yếu trước distribution shift, phụ thuộc mạnh vào dữ liệu huấn luyện.

Ý nghĩa cho nghiên cứu tương lai là cần phát triển các phương pháp robust hơn, cải thiện khả năng generalization, đồng thời tích hợp các kỹ thuật xử lý obfuscation và thích ứng với concept drift trong môi trường thực tế.

Is this conversation helpful so far?

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#)

2.2. Tác vụ 2 – Giải thích khái niệm Gradient Descent

Cấp độ	Prompt
Cơ bản	"Giải thích gradient descent là gì."
Cải tiến	"Giải thích khái niệm Gradient Descent trong machine learning cho sinh viên năm nhất ngành CNTT. Yêu cầu: (1) Dùng ngôn ngữ đơn giản, tránh công thức phức tạp, (2) Đưa ra ít nhất 1 ví dụ trực quan từ đời thực, (3) Giải thích tại sao learning rate quan trọng, (4) Nêu 2 biến thể phổ biến (SGD, Mini-batch). Tiếng Việt."
Nâng cao	"Bạn là một giảng viên đại học giỏi nhất trong việc giải thích các khái niệm phức tạp một cách đơn giản. Hãy giải thích Gradient Descent cho sinh viên năm nhất CNTT theo cách sau: Bước 1: Bắt đầu bằng một analogy từ đời thường (ví dụ: tìm đường xuống núi khi bị bịt mắt). Bước 2: Từ analogy, dẫn dắt sang khái niệm kỹ thuật. Bước 3: Giải thích learning rate bằng cách thay đổi analogy (bước đi lớn vs nhỏ). Bước 4: So sánh Batch GD, SGD, Mini-batch GD bằng bảng. Bước 5: Kết thúc bằng 3 câu hỏi kiểm tra hiểu biết. Tiếng Việt, giữ thuật ngữ tiếng Anh."

Kỹ thuật áp dụng ở prompt nâng cao: Role prompting (giảng viên), chain-of-thought (hướng dẫn từng bước), structured output (bảng so sánh), self-verification (câu hỏi kiểm tra).

ChatGPT

Giải thích gradient descent là gì

Share

Gradient Descent là một thuật toán tối ưu dùng trong Machine Learning để tìm giá trị nhỏ nhất (minimum) của một hàm, thường là loss function (hàm mất mát).

Ý tưởng trực quan

Hãy tưởng tượng bạn đang đứng trên một ngọn đồi và muốn đi xuống điểm thấp nhất:

- Bạn nhìn độ dốc xung quanh
- Đi theo hướng dốc xuống nhất
- Lặp lại cho đến khi không thể xuống thêm

Gradient Descent làm đúng điều đó nhưng trên không gian toán học.

Nguyên lý hoạt động

Tại mỗi bước, tham số được cập nhật theo công thức:

$$\theta = \theta - \eta \nabla J(\theta)$$

Trong đó:

+ Ask anything

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).

Quy trình

- Khởi tạo tham số (random)
- Tính gradient
- Cập nhật tham số
- Lặp lại đến khi hội tụ

Các biến thể phổ biến

- Batch Gradient Descent: dùng toàn bộ dữ liệu
- Stochastic Gradient Descent (SGD): dùng từng điểm dữ liệu
- Mini-batch Gradient Descent: dùng một nhóm (phổ biến nhất)

+ Ask anything

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).

[illegible]

ChatGPT

- Học nhanh
- Không bị "lạc đường"

Hai biến thể phổ biến

- Stochastic Gradient Descent (SGD)**
 - Cập nhật sau mỗi dữ liệu
 - Nhanh nhưng hơi "nhiều" (lúc lên lúc xuống)
- Mini-batch Gradient Descent**
 - Cập nhật theo một nhóm nhỏ dữ liệu
 - Cân bằng giữa tốc độ và độ ổn định
 - Đây là cách dùng phổ biến nhất hiện nay

Tóm lại

Gradient Descent là cách để mô hình tự sửa sai từng bước nhỏ, giống như đi xuống dốc để tìm điểm thấp nhất (sai số nhỏ nhất).

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#)

ChatGPT

Bạn là một giảng viên đại học giỏi nhất trong việc giải thích các khái niệm phức tạp một cách đơn giản. Hãy giải thích Gradient Descent cho sinh viên năm nhất CNTT theo cách sau: Bước 1: Bắt đầu bằng một analogy từ đời thường (ví dụ: tìm đường xuống núi khi bị bịt mắt). Bước 2: Từ analogy, dẫn dắt sang khái niệm kỹ thuật. Bước 3: Giải thích learning rate bằng cách thay đổi analogy (bước đi lớn vs nhỏ). Bước 4: So sánh Batch GD, SGD, Mini-batch GD bằng bảng. Bước 5: Kết thúc bằng 3 câu hỏi kiểm tra hiểu biết. Tiếng Việt, giữ thuật ngữ tiếng Anh.

Bước 1: Analogy đời thường

Hãy tưởng tượng bạn đang đứng trên một ngọn núi, bị bịt mắt, và nhiệm vụ là đi xuống điểm thấp nhất (thung lũng).

Bạn không nhìn thấy toàn cảnh, nên mỗi lần bạn sẽ:

- Dùng chân "cảm nhận" độ dốc xung quanh
- Chọn hướng dốc xuống nhất
- Đi một bước theo hướng đó
- Lặp lại cho đến khi không thể xuống thêm

Đây chính là trực giác của Gradient Descent.

Bước 2: Từ analogy → khái niệm kỹ thuật

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#)

ChatGPT

Bước 2: Từ analogy → khái niệm kỹ thuật

Trong Machine Learning:

- "Ngọn núi" = loss function (hàm sai số)
- "Vị trí của bạn" = các tham số (parameters) của mô hình
- "Độ dốc" = gradient (đạo hàm)

Gradient Descent là cách:

- Tính gradient để biết hướng sai số tăng nhanh nhất
- Đi ngược lại hướng đó để giảm sai số
- Lặp lại nhiều lần → tìm được điểm sai số nhỏ nhất (optimal)

Bước 3: Learning rate (độ dài bước chân)

Quay lại ví dụ leo núi:

- Nếu bạn bước quá dài → có thể đi quá điểm thấp nhất, rồi lại phải quay lại (dao động)
- Nếu bước quá ngắn → đi rất lâu mới xuống tới nơi

Learning rate chính là độ dài bước chân:

- Lớn → nhanh nhưng dễ sai
- Nhỏ → chính xác hơn nhưng chậm

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#)

ChatGPT

Bước 4: So sánh các biến thể

Phương pháp	Cách cập nhật	Ưu điểm	Nhược điểm
Batch Gradient Descent	Dùng toàn bộ dữ liệu rồi mới cập nhật	Ổn định, chính xác	Rất chậm nếu dữ liệu lớn
Stochastic Gradient Descent (SGD)	Cập nhật sau từng mẫu dữ liệu	Nhanh, linh hoạt	Nhiều, dao động mạnh
Mini-batch Gradient Descent	Cập nhật theo nhóm nhỏ dữ liệu	Cân bằng tốc độ & ổn định	Cần chọn kích thước batch phù hợp

👉 Trong thực tế, Mini-batch GD được dùng nhiều nhất.

? Bước 5: Câu hỏi kiểm tra

1. Nếu learning rate quá lớn thì điều gì có thể xảy ra khi dùng Gradient Descent?
2. Tại sao SGD lại nhanh hơn Batch Gradient Descent nhưng kém ổn định hơn?
3. Trong analogy "xuống núi", gradient tương ứng với điều gì?

👉 Nếu bạn trả lời được 3 câu này, bạn đã hiểu khá chắc Gradient Descent rồi.

+ Ask anything

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#)

2.3. Tác vụ 3 – Tạo bộ câu hỏi ôn tập (CTDL & GT)

Cấp độ	Prompt
Cơ bản	"Tạo câu hỏi ôn tập về stack, queue và linked list."
Cải tiến	"Tạo bộ 15 câu hỏi ôn tập về cấu trúc dữ liệu: Stack, Queue, Linked List. Yêu cầu: (1) 5 câu trắc nghiệm (có 4 đáp án, chỉ rõ đáp án đúng), (2) 5 câu đúng/sai (có giải thích), (3) 5 câu tự luận ngắn. Phân bổ đều 3 chủ đề. Mức độ: từ nhớ kiến thức đến vận dụng. Tiếng Việt."
Nâng cao	"Bạn là giảng viên môn Cấu trúc dữ liệu & Giải thuật tại một trường đại học CNTT. Hãy tạo bộ đề ôn tập 15 câu hỏi theo Bloom's Taxonomy: Mức 1 – Nhớ (3 câu trắc nghiệm): Kiểm tra định nghĩa, đặc điểm cơ bản. Ví dụ: 'Stack hoạt động theo nguyên tắc nào? A. FIFO B. LIFO C. Random D. Priority'. Mức 2 – Hiểu (4 câu đúng/sai có giải thích): Kiểm tra hiểu bản chất. Mức 3 – Vận dụng (4 câu): Cho đoạn code, yêu cầu trace output. Mức 4 – Phân tích (2 câu): So sánh hiệu năng, chọn CTDL phù hợp cho bài toán cụ thể. Mức 5 – Sáng tạo (2 câu): Thiết kế giải pháp dùng CTDL cho bài toán thực tế. Kèm đáp án và giải thích chi tiết. Tiếng Việt, code bằng Python."

Kỹ thuật áp dụng ở prompt nâng cao: Role prompting (giảng viên), few-shot example (câu hỏi mẫu), structured output (phân theo Bloom's Taxonomy), constraint specification (ngôn ngữ, format, số lượng).

ChatGPT

Tạo câu hỏi ôn tập về stack, queue và linked list

Share

Dưới đây là bộ câu hỏi ôn tập từ cơ bản → nâng cao về Stack, Queue và Linked List:

Phần 1: Câu hỏi lý thuyết cơ bản

1. Stack là gì? Nguyên lý hoạt động của Stack (LIFO) là gì?

2. Queue là gì? Khác gì với Stack (FIFO vs LIFO)?

3. Linked List là gì? Khác gì với mảng (array)?

4. Phân biệt Singly Linked List và Doubly Linked List.

5. Các phép toán cơ bản trên:

- Stack
- Queue
- Linked List

Phần 2: Hiểu và so sánh

6. Khi nào nên dùng Stack thay vì Queue?

7. Tại sao Linked List chèn/xóa nhanh hơn Array trong một số trường hợp?

8. Nhược điểm của Linked List so với Array là gì? ↓

9. Queue có thể cài đặt bằng Stack không? Giải thích ý tưởng.

+ Ask anything

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#)

ChatGPT

Share

8. Nhược điểm của Linked List so với Array là gì?

9. Queue có thể cài đặt bằng Stack không? Giải thích ý tưởng.

10. Stack có thể dùng để giải quyết những bài toán nào trong thực tế?

Phần 3: Truy vết (trace) thuật toán

11. Cho Stack rỗng, thực hiện các thao tác:
`push(1), push(2), pop(), push(3), pop()`
→ Kết quả cuối cùng là gì?

12. Cho Queue rỗng:
`enqueue(1), enqueue(2), dequeue(), enqueue(3)`
→ Phần tử đầu (front) là gì?

13. Với Linked List:
1 → 2 → 3 → 4
Nếu xóa node có giá trị 2 thì danh sách còn gì?

Phần 4: Bài tập vận dụng

14. Viết thuật toán kiểm tra chuỗi ngoặc hợp lệ bằng Stack.

15. Đảo ngược một Linked List.

16. Cài đặt Queue bằng Linked List.

17. Tìm phần tử ở giữa của Linked List.

18. Kiểm tra 1 linked List có phải palindrome không.

16. Cài đặt Queue bằng Linked List.

17. Tìm phần tử ở giữa của Linked List.

18. Kiểm tra Linked List có phải palindrome không.

Phần 5: Câu hỏi nâng cao (tư duy)

19. Tại sao Stack thường được dùng trong đệ quy (recursion)?

20. Trong hệ điều hành, Queue được dùng ở đâu?

21. Nếu bộ nhớ bị giới hạn, bạn chọn Array hay Linked List? Vì sao?

22. Làm thế nào để phát hiện vòng lặp (cycle) trong Linked List?

Nếu bạn muốn, mình có thể:

Tạo đề kiểm tra trắc nghiệm (MCQ)

Hoặc giải chi tiết từng câu để bạn luyện thi

Ask anything

Share

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).

Tạo bộ 15 câu hỏi ôn tập về cấu trúc dữ liệu: Stack, Queue, Linked List.

Yêu cầu: (1) 5 câu trắc nghiệm (có 4 đáp án, chỉ rõ đáp án đúng), (2) 5 câu đúng/sai (có giải thích), (3) 5 câu tự luận ngắn. Phân bố đều 3 chủ đề. Mức độ: từ nhớ kiến thức đến vận dụng. Tiếng Việt.

Dưới đây là bộ 15 câu hỏi ôn tập về Stack, Queue, Linked List (mức độ từ nhớ → vận dụng):

Phần 1: Trắc nghiệm (5 câu)

Câu 1 (Stack)
Nguyên lý hoạt động của Stack là:
A. FIFO
B. LIFO
C. Random access
D. Circular
Đáp án: B

Câu 2 (Queue)
Thao tác nào dùng để thêm phần tử vào Queue?
A. push

ChatGPT

Phản 3: Tự luận ngắn (5 câu)

Câu 11 (Stack)
Giải thích cách sử dụng Stack để kiểm tra chuỗi dấu ngoặc hợp lệ.

Câu 12 (Queue)
Mô tả cách hoạt động của Queue trong hệ điều hành (ví dụ: hàng đợi tiến trình).

Câu 13 (Linked List)
Trình bày cách chèn một node vào giữa Singly Linked List.

Câu 14 (So sánh)
So sánh ưu và nhược điểm của Linked List và Array.

Câu 15 (Vận dụng)
Mô tả thuật toán tìm phần tử ở giữa của Linked List mà không cần biết độ dài trước.

Nếu bạn muốn, mình có thể:

+ Ask anything

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#)

Câu 13 (Linked List)
Trình bày cách chèn một node vào giữa Singly Linked List.

Câu 14 (So sánh)
So sánh ưu và nhược điểm của Linked List và Array.

Câu 15 (Vận dụng)
Mô tả thuật toán tìm phần tử ở giữa của Linked List mà không cần biết độ dài trước.

Nếu bạn muốn, mình có thể:

- Tạo đáp án chi tiết phân tự luận
- Hoặc nâng cấp thành đề thi giữa kỳ / cuối kỳ hoàn chỉnh

+ Ask anything

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#)

Bạn là giảng viên môn Cấu trúc dữ liệu & Giải thuật tại một trường đại học CNTT. Hãy tạo bộ đề ôn tập 15 câu hỏi theo Bloom's Taxonomy: Mức 1 – Nhớ (3 câu trắc nghiệm): Kiểm tra định nghĩa, đặc điểm cơ bản. Ví dụ: 'Stack hoạt động theo nguyên tắc nào? A. FIFO B. LIFO C. Random D. Priority'. Mức 2 – Hiểu (4 câu đúng/sai có giải thích): Kiểm tra hiểu bản chất. Mức 3 – Vận dụng (4 câu): Cho đoạn code, yêu cầu trace output. Mức 4 – Phân tích (2 câu): So sánh hiệu năng, chọn CTDL phù hợp cho bài toán cụ thể. Mức 5 – Sáng tạo (2 câu): Thiết kế giải pháp đúng CTDL cho bài toán thực tế. Kèm đáp án và giải thích chi tiết. Tiếng Việt, code bằng Python.

Dưới đây là bộ đề ôn tập 15 câu theo Bloom's Taxonomy cho chủ đề Stack, Queue, Linked List:

Mức 1 – Nhớ (3 câu trắc nghiệm)

Câu 1 (Stack)
Stack hoạt động theo nguyên tắc nào?
A. FIFO
B. LIFO
C. Random
D. Priority

+ Ask anything

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#)

ChatGPT

👉 Đáp án: B

Giải thích: Stack = Last In First Out (vào sau ra trước).

Câu 2 (Queue)

Thao tác lấy phần tử ra khỏi Queue gọi là gì?

A. push

B. pop

C. dequeue

D. removeLast

👉 Đáp án: C

Giải thích: dequeue lấy phần tử ở đầu hàng.

Câu 3 (Linked List)

Linked List có ưu điểm chính nào so với Array?

A. Truy cập ngẫu nhiên nhanh

B. Bộ nhớ liên tục

C. Chèn/xóa linh hoạt

D. Tốc độ luôn nhanh hơn

👉 Đáp án: C

Giải thích: Không cần dịch chuyển phần tử khi chèn/xóa.

+ Ask anything

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#)

ChatGPT

D. Tốc độ luôn nhanh hơn

👉 Đáp án: C

Giải thích: Không cần dịch chuyển phần tử khi chèn/xóa.

📌 Mức 2 – Hiểu (4 câu đúng/sai)

Câu 4

Stack có thể dùng để thực hiện đệ quy.

👉 Đúng

Giải thích: Ngăn xếp lời gọi hàm (call stack) lưu trạng thái khi đệ quy.

Câu 5

Queue luôn thêm và xóa ở cùng một đầu.

👉 Sai

Giải thích: Thêm ở rear, xóa ở front.

Câu 6

Linked List luôn tiết kiệm bộ nhớ hơn Array.

👉 Sai

Giải thích: Mỗi node cần thêm con trỏ → tốn bộ nhớ hơn.

+ Ask anything

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#)

ChatGPT

Câu 6

Linked List luôn tiết kiệm bộ nhớ hơn Array.

👉 Sai

Giải thích: Mỗi node cần thêm con trỏ → tốn bộ nhớ hơn.

Câu 7

Mini-batch processing trong Queue không tồn tại.

👉 Sai

Giải thích: Queue vẫn có thể xử lý theo lô (batch), đây là cách sử dụng chứ không phải giới hạn của cấu trúc.

📌 Mức 3 – Vận dụng (4 câu trace code – Python)

Câu 8 (Stack)

```
Python
stack = []
stack.append(1)
stack.append(2)
stack.pop()
stack.append(3)
print(stack)
```

👉 Đáp án: [1, 3]

Giải thích: Theo dõi các thao tác push và pop trên stack.

+ Ask anything

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#)

Trang 14

ChatGPT

Python

```
print(stack)
```

Run

Đáp án: [1, 3]

Giải thích: pop() loại 2 → còn [1], thêm 3.

Câu 9 (Queue)

Python

```
from collections import deque
q = deque()
q.append(1)
q.append(2)
q.popleft()
q.append(3)
print(list(q))
```

Run

Đáp án: [2, 3]

Giải thích: popleft() loại 1.

Câu 10 (Linked List đơn giản)

Ask anything

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#)

Giải thích: a.next là b, đã đổi giá trị thành 5.

Câu 11 (Stack nâng cao)

Python

```
stack = []
for i in range(3):
    stack.append(i)

while stack:
    print(stack.pop(), end=" ")
```

Run

Đáp án: 2 1 0

Giải thích: LIFO → đảo ngược thứ tự.

Mức 4 – Phân tích (2 câu)

Câu 12

So sánh Array và Linked List khi chèn phần tử vào giữa.

Đáp án:

- Array: $O(n)$ do phải dịch chuyển phần tử
- Linked List: $O(1)$ nếu đã có con trỏ tới vị trí

Kết luận: Linked List hiệu quả hơn cho chèn/xóa.

Câu 13

Bài toán: Xử lý hàng đợi in ấn (printer queue). Nên dùng cấu trúc nào?

Đáp án: Queue

Giải thích:

- Công việc đến trước xử lý trước → FIFO
- Phù hợp với mô hình hàng đợi thực tế

Mức 5 – Sáng tạo (2 câu)

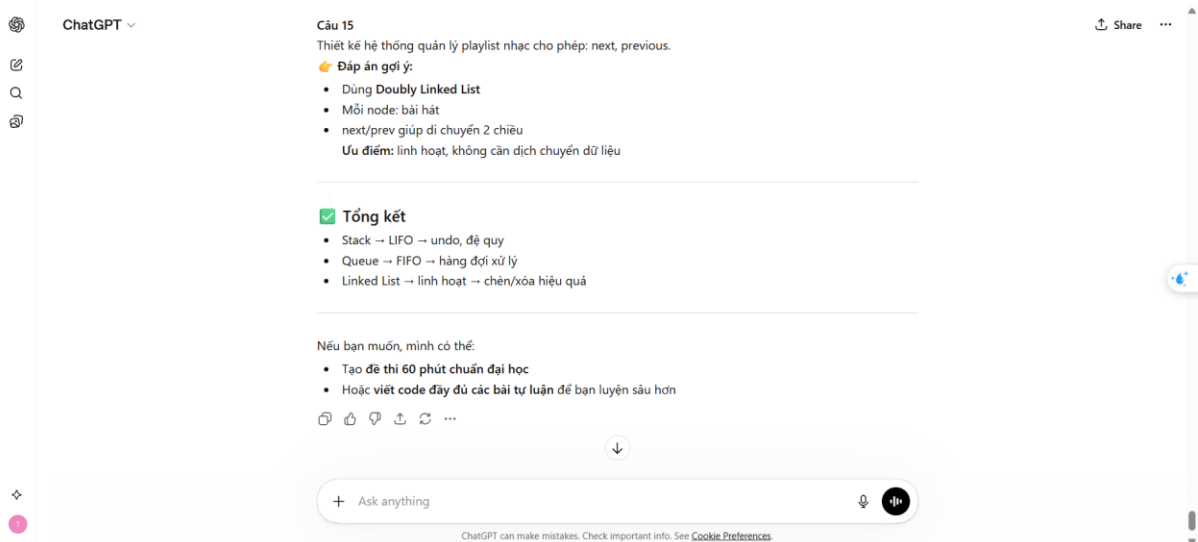
Câu 14

Thiết kế hệ thống undo/redo trong trình soạn thảo.

Đáp án gợi ý:

- Dùng 2 Stack:
 - Undo stack
 - Redo stack
- Khi thao tác mới → push vào undo, xóa redo
- Undo → chuyển từ undo → redo

Trang 15



3. So sánh và phân tích kết quả

3.1. Bảng so sánh tổng hợp

Đánh giá trên thang 10 điểm cho mỗi tiêu chí:

	Chính xác	Chi tiết	Cấu trúc	Phù hợp ngữ cảnh	Sáng tạo	TB
TV1 – Cơ bản	6	5	4	5	4	4.8
TV1 – Cải tiến	8	8	8	7	6	7.4
TV1 – Nâng cao	9	9	9	9	8	8.8
TV2 – Cơ bản	7	5	5	4	4	5.0
TV2 – Cải tiến	8	8	7	8	7	7.6
TV2 – Nâng cao	9	9	9	9	9	9.0
TV3 – Cơ bản	6	4	3	4	3	4.0
TV3 – Cải tiến	8	8	8	7	7	7.6
TV3 – Nâng cao	9	9	10	9	9	9.2

Nhận xét: Điểm trung bình tăng rõ rệt từ cơ bản (4.6) → cải tiến (7.5) → nâng cao (9.0). Mức cải thiện lớn nhất nằm ở bước từ cơ bản lên cải tiến (tăng 63%), cho thấy chỉ cần thêm cấu trúc và yêu cầu cụ thể đã cải thiện đáng kể chất lượng đầu ra.

3.2. Phân tích tại sao prompt nâng cao hiệu quả hơn

Lý do 1 – Role prompting giúp AI "nhập vai" đúng: Khi được gán vai "nghiên cứu sinh" hay "giảng viên", AI điều chỉnh giọng văn, mức độ chuyên sâu và cách trình bày phù hợp hơn. Ví dụ: ở Tác vụ 2, prompt cơ bản cho giải thích quá kỹ thuật, trong khi prompt nâng cao (vai giảng viên giải thích cho SV năm nhất) cho analogy trực quan và dễ hiểu.

Lý do 2 – Structured output giảm sự mơ hồ: Khi prompt chỉ rõ format đầu ra (bảng, danh sách, theo thứ tự bước), AI tuân theo cấu trúc nhất quán thay vì viết tự do. Prompt cơ bản cho Tác vụ 3 tạo ra 7 câu hỏi lộn xộn; prompt nâng cao tạo đúng 15 câu phân theo Bloom's Taxonomy.

Lý do 3 – Few-shot examples đặt "chuẩn" chất lượng: Ví dụ mẫu trong prompt giúp AI hiểu chính xác mức độ chi tiết, format và giọng văn mong muốn. Không có ví dụ, AI phải "đoán" và thường chọn cách an toàn nhất (chung chung).

Lý do 4 – Chain-of-thought tạo logic mạch lạc: Hướng dẫn AI suy nghĩ theo từng bước (Bước 1 → Bước 2 → ...) giúp output có logic tốt hơn, đặc biệt hiệu quả ở Tác vụ 2 (giải thích khái niệm phức tạp).

Lý do 5 – Constraints cụ thể ngăn AI "lan man": Giới hạn rõ (200 từ, 15 câu, tiếng Việt, giữ thuật ngữ Anh) giúp AI tập trung vào đúng yêu cầu thay vì tự mở rộng không kiểm soát.

4. Tổng hợp nguyên tắc viết prompt hiệu quả

Dựa trên kết quả thử nghiệm, tôi rút ra 7 nguyên tắc:

#	Nguyên tắc	Giải thích & Ví dụ từ bài tập
1	Xác định rõ vai trò cho AI	Gán vai cụ thể (giảng viên, nghiên cứu sinh, chuyên gia) giúp AI điều chỉnh giọng văn và độ sâu. VD: "Bạn là giảng viên giỏi nhất..." → giải thích dễ hiểu hơn.
2	Chỉ rõ format đầu ra	Mô tả cấu trúc mong muốn: bảng, danh sách, bước 1-2-3, hoặc template cụ thể. VD: "Theo format: [Mục tiêu] → [Phương pháp] → [Kết quả]".
3	Đặt constraints cụ thể	Giới hạn độ dài, ngôn ngữ, số lượng, mức độ chuyên sâu. VD: "200 từ, tiếng Việt, giữ thuật ngữ tiếng Anh, 15 câu hỏi".
4	Cung cấp ví dụ mẫu	1–2 ví dụ output mong muốn giúp AI "thấy" chuẩn chất lượng. VD: "Ví dụ output: 'Nghiên cứu đề xuất mô hình XYZ...'".
5	Hướng dẫn suy nghĩ từng bước	Chia nhiệm vụ phức tạp thành các bước rõ ràng. VD: "Bước 1: Bắt đầu bằng analogy → Bước 2: Dẫn sang khái niệm kỹ thuật → ...".
6	Xác định đối tượng đọc	Nêu rõ ai sẽ đọc kết quả để AI điều chỉnh mức độ. VD: "cho sinh viên năm nhất CNTT" vs "cho chuyên gia" → kết quả rất khác.
7	Lặp lại và cải tiến (iterate)	Prompt tốt nhất thường là kết quả sau 2–3 lần chỉnh sửa. Đọc output, nhận diện thiếu sót, bổ sung yêu cầu vào prompt tiếp theo.

5. Kết luận

Kết quả thử nghiệm cho thấy kỹ năng viết prompt có tác động quyết định đến chất lượng đầu ra của AI. Điểm trung bình tăng từ 4.6 (cơ bản) lên 9.0 (nâng cao) – gần gấp đôi – chỉ bằng cách thay đổi cách đặt câu hỏi. Điều đáng chú ý là bước cải thiện lớn nhất nằm ở việc chuyển từ prompt "hỏi chung chung" sang prompt "có cấu trúc và yêu cầu cụ thể" — một kỹ năng mà bất kỳ sinh viên nào cũng có thể học và áp dụng ngay.

Prompt engineering không phải là "hack" hay "trick" — đó là kỹ năng giao tiếp rõ ràng với máy, và bản chất cũng chính là kỹ năng diễn đạt ý tưởng mạch lạc — một năng lực quan trọng trong mọi lĩnh vực.